Certificate Management System v1.3

Project Submission Document

Submission date: 18 September 2026
Project type: Department certificate and On-Duty management web application
Status: Implemented and locally validated

1. Executive Summary

The Certificate Management System is a full-stack web application for registering On-Duty (OD) participation, collecting student certificates, identifying missing submissions, and generating monthly reports. It replaces fragmented spreadsheet and file-sharing workflows with a role-aware application backed by PostgreSQL.

The system has four application roles:

- Student: views personal details and OD records, submits certificates, and downloads submitted certificates.
- Student Representative: searches students and registers OD records for events.
- Coordinating Faculty: reviews OD records, identifies missing certificates, and generates reports.
- Administrator: provisions and manages the role hierarchy through administrative APIs.

2. Objectives

1. Provide a central record for OD registrations.
2. Generate unique OD registration numbers safely under concurrent use.
3. Allow students to submit certificates against their own OD records.
4. Prevent unauthorized users from viewing or changing other students' records.
5. Provide faculty with monthly summaries, Excel exports, and certificate ZIP downloads.
6. Store certificate files in a predictable, protected filesystem structure.

3. Technology Stack

Layer	Technology
Frontend	React 18, Vite, React Router, Axios, React Hook Form, Tailwind CSS
Backend	Node.js, Express.js, ES modules
Database	PostgreSQL 18, pg connection pool
Authentication	JWT tokens and bcrypt password hashes
File upload	Multer with memory buffering and server-side validation
Reports	ExcelJS for .xlsx, Adm-Zip for .zip
Testing	Jest and Supertest-style backend tests; Vitest and Testing Library frontend tests

4. Architecture

The application uses a layered client-server architecture:

text
React/Vite frontend
|
| Axios JSON and multipart requests
v
Express REST API
|
+-- JWT authentication and role authorization
+-- Models for PostgreSQL access
+-- File storage and report generation
v
PostgreSQL database + certificate filesystem storage

The frontend contains role-specific pages, protected routes, an authentication context, and service modules. The backend contains route modules, middleware, models, migrations, configuration, and utilities. PostgreSQL provides foreign keys, unique constraints, indexes, and the OD registration sequence.

5. Main Workflows

Student workflow

1. Log in with institutional register number and date of birth.
2. View personal profile and OD records.
3. Select an owned OD registration number.
4. Upload a PDF, JPG, JPEG, or PNG certificate up to 5 MB.
5. View or download submitted certificates.

Student Representative workflow

1. Log in with a provisioned staff credential.
2. Search students by register number, name, or department.
3. Register an OD record with student ID and event details.
4. Receive a generated number such as OD-0001.
5. View OD records available to the representative role.

Coordinating Faculty workflow

1. Log in with a provisioned faculty credential.
2. View OD records and certificate status.
3. Request a monthly summary using YYYY-MM.
4. Export the filtered records to Excel.
5. Download submitted certificates for a month as a ZIP archive.

Administrator workflow

1. Log in with an administrator credential.
2. Provision coordinating faculty accounts.
3. Support the faculty-to-representative role hierarchy.

6. Database Design

The database is created by three migrations:

- 001initialschema.sql: base students, users, OD records, certificates, sequence, indexes, and foreign keys.
- 002authenticationandrolehierarchy.sql: password hashes, faculty assignment, role constraints, and authentication indexes.
- 003adminroleandstudentdob.sql: administrator role support and student date of birth.

Core tables:

- students: institutional register number, name, department, year, section, and date of birth.
- users: username, role, password hash, active state, student link, and assigned faculty link.
- odrecords: student link, generated registration number, event details, creator, status, and timestamps.
- certificates: one-to-one OD certificate metadata, secure path, file type, size, and reporting month.

The registrationnumberseq PostgreSQL sequence produces globally unique numbers formatted as OD-0001, OD-0002, and so on. Sequence gaps after failed transactions are expected and do not compromise uniqueness.

7. API Summary

Base URL: http://localhost:5000/api

Route group	Purpose
/auth	Login, logout, and current user identity
/users	Faculty and representative account administration
/students	Student profile, search, personal OD records, and certificates
/od-records	OD creation and OD record access
/certificates	Certificate upload, listing, and download
/reports	Monthly summary, Excel export, and ZIP export

The health endpoint is GET /api/health. Protected endpoints require Authorization: Bearer <JWT>.

8. Security and Validation

- JWT authentication is required for protected routes.
- Role checks are enforced on the server, not only in the frontend.
- Student ownership is checked before certificate submission and download.
- Passwords are stored as bcrypt hashes.
- SQL uses parameterized queries through pg.
- Uploads allow only PDF, JPG, JPEG, and PNG files up to 5 MB.
- Certificate files are stored outside the public frontend directory.
- Production configuration requires a strong JWT secret, explicit CORS origins, configured storage, and HTTPS.
- Invalid tokens, missing roles, invalid files, missing records, and duplicate certificates receive centralized API errors.

9. Local Setup

Prerequisites

- Node.js LTS
- PostgreSQL 12 or later; the validated environment uses PostgreSQL 18
- npm

Database and backend

powershell
cd backend
npm install
copy .env.example .env
npm run migrate
npm start

Backend URL: http://localhost:5000
Health URL: http://localhost:5000/api/health

Frontend

powershell
cd frontend
npm install
copy .env.example .env
npm run dev -- --host localhost

Frontend URL: http://localhost:5173

10. Local Validation Accounts

These credentials are for local demonstration and testing only:

Role	Username	Password
Admin	admin	Admin1234
Student	22AD001	2004-06-15
Student Representative	rep1	Rep12345
Coordinating Faculty	faculty1	Faculty123

Credentials must be changed or removed before any production deployment.

11. Verification Evidence

The implementation was validated against a live local PostgreSQL database and running HTTP services:

- PostgreSQL server was running and the certificatemanagement database was reachable.
- All three database migrations completed successfully.

- GET /api/health returned status OK.
- Admin, student, and student representative login plus /auth/me flows succeeded.
- A representative created an OD record with registration number OD-0001.
- A student uploaded 22AD001OD-0001.pdf successfully.
- Faculty monthly summary returned one OD record, one submitted certificate, and zero missing certificates.
- Excel export returned a valid .xlsx response with a nonzero payload.
- ZIP export returned a valid .zip response with a nonzero payload.
- Backend tests passed: 2 suites and 3 tests.
- Frontend tests passed: 1 suite and 3 tests.
- Frontend production build completed successfully.

12. Project Structure

text
backend/ Express API, models, routes, middleware, migrations, tests
frontend/ React/Vite SPA, pages, role components, services, tests
docs/ API, database, design, and submission documentation
storage/ Certificate files organized by year and month
README.md Project overview and startup guide

13. Scope and Limitations

The current release intentionally does not include event discovery, certificate approval or rejection, participation verification, automatic OD blocking, QR codes, OCR, payments, or multi-tenant administration. Department policy decisions remain open for production authentication policy, OD cancellation, late submissions, department scope, and editing authority.

14. Conclusion

Certificate Management System v1.3 provides a working, secure foundation for departmental OD and certificate administration. The core workflows, role boundaries, database integrity, file handling, and reporting functions are implemented and verified locally through automated tests and live API validation.